FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main()
{
    int m,n,i,j;

    printf("Enter number of memory blocks:\n");
    scanf("%d",&m);

    int block[m], remain[m];

    printf("Enter size of each block:\n");
    for(i=0;i<m;i++)
    {
        scanf("%d",&block[i]);
        remain[i] = block[i];
    }

    printf("Enter number of processes:\n");
    scanf("%d",&n);

    int process[n];

    printf("Enter size of each process:\n");
```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```
    printf("Enter size of each process:\n");
    for(i=0;i<n;i++)
        scanf("%d",&process[i]);

    for(i=0;i<n;i++)
    {
        int allocated = 0;

        for(j=0;j<m;j++)
        {
            if(remain[j] >= process[i])
            {
                remain[j] -= process[i];
                int fragment = remain[j];

                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                       i+1,process[i],j+1,block[j],fragment);

                allocated = 1;
                break;
            }
        }

        if(!allocated)
```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```
31
32          for(j=0;j<m;j++)
33          {
34              if(remain[j] >= process[i])
35              {
36                  remain[j] -= process[i];
37                  int fragment = remain[j];
38
39                  printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
40                         i+1,process[i],j+1,block[j],fragment);
41
42                  allocated = 1;
43                  break;
44              }
45          }
46
47          if(!allocated)
48          {
49              printf("Process %d of size %d is not allocated\n",i+1,process[i]);
50          }
51      }
52
53      return 0;
54  }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test

Success

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
```

Test Case - 1 Success

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
```

Test Case - 2 Success

Input :

```
4
50 100 200 300
3
70 90 150
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 70 is allocated to Block 2 of size 100 with Fragment 30
Process 2 of size 90 is allocated to Block 3 of size 200 with Fragment 110
Process 3 of size 150 is allocated to Block 4 of size 300 with Fragment 150
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 70 is allocated to Block 2 of size 100 with Fragment 30
Process 2 of size 90 is allocated to Block 3 of size 200 with Fragment 110
```